

MLOps Assignment Report

Q1 — Technical Debt Diagnosis

(a) **Entanglement (CACE principle).** The delivery-time feature and the favorite-restaurants feature share the same underlying model, so “Changing Anything Changes Everything” — a rounding-logic tweak shifted the feature’s distribution and silently altered learned weights/interactions for an unrelated output.

(b) **Undeclared Consumers.** The marketing dashboard reads the raw output table without the ML team’s knowledge, creating an invisible dependency that constrains any future change to the output schema or serving pipeline.

(c) **Pipeline Jungles / Glue Code.** Fourteen chained, undocumented shell scripts with no orchestration is the classic “pipeline jungle” — brittle, unversioned glue code that is costly to modify, test, or debug.

Mitigation (for c): Replace the shell-script chain with a workflow orchestration tool such as Apache Airflow (or Kubeflow Pipelines). Express the 14 steps as a versioned DAG with explicit task dependencies, retries, logging, and lineage tracking; this makes the pipeline testable, self-documenting, and auditable, directly removing the pipeline-jungle debt.

Q2 — MLP Hyperparameter Experiments on MNIST (via MLflow)

Six MLPClassifier runs (`hidden_layer_sizes=(128,)`) were logged to MLflow, varying `learning_rate` $\in \{0.001, 0.01\}$ and `batch_size` $\in \{32, 64, 128\}$, on MNIST (56,700 train / 6,300 val / 7,000 test).

Analysis (150–250 words):

The best-performing run is LR=0.001, Batch=32 (test accuracy 0.9757), followed closely by LR=0.001 with Batch=64/128 (0.9737 each). Lower learning rate combined with smaller batches gives more, noisier-but-finer gradient updates per epoch, letting the model converge to a better minimum within 10 epochs. Overfitting evidence: for LR=0.001/Batch=32, `train_loss` decreases monotonically to ~ 0.02 by epoch 10, while `val_loss` bottoms out near epoch 5 (~ 0.095) and then drifts up to ~ 0.109 — the widening train/val gap after epoch 5 is a clear overfitting signal. The LR=0.001/Batch=128 run shows a similar but milder pattern, with `val_loss` staying flatter (~ 0.096 – 0.097) late in training, suggesting slightly better generalization despite marginally higher `train_loss`. The LR=0.01 runs (all batch sizes) never reach the low `train_loss` of the LR=0.001 runs and show noisy, rising validation loss from early epochs — indicating instability rather than useful learning, not classic overfitting. Comparing hyperparameters: test accuracy is clearly stratified by learning rate (0.001 group: 0.973–0.976; 0.01 group: 0.956–0.968), while batch size only produces a ~ 0.002 – 0.010 spread within each learning-rate group. Learning rate is therefore the dominant hyperparameter for this task, with batch size a secondary, fine-tuning factor.